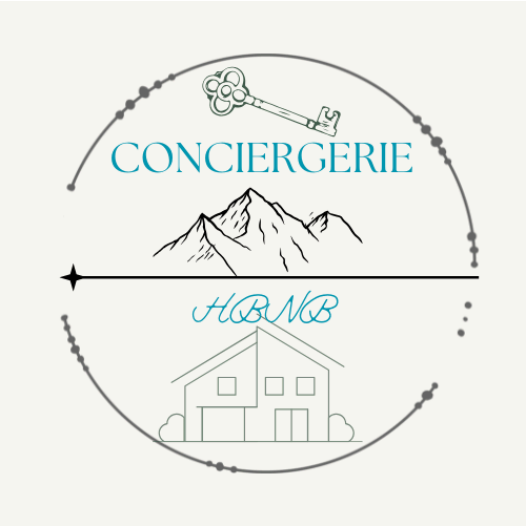


Documentation technique : Projet Hbnb pour Holberton Scholl

Auteur: Abdelrahman Azrhar, Philippe Lapique

Sommaire:

1. **Introduction du projet.**
   1.1 Objectif du document
   1.2 Présentation des couches
   1.3 Hbnb, c'est quoi ?

2. **Architecture Globale.**
   2.1 Description de la structure
   2.2 Pattern Facade et justification
   2.3 Diagramme d'architecture

3. **Logique Métier (Business Logic Layer).**
   3.1 Description des classes principales
   3.2 Cardinalités et relations
   3.3 Diagramme de classes UML

4. **Appels des API (API Interaction Flow).**
   4.1 Méthodologie (diagramme de séquence)
   4.2 Cas d'utilisation : Enregistrement utilisateur
   4.3 Cas d'utilisation : Création d'une annonce
   4.4 Cas d'utilisation : Soumission d'un avis
   4.5 Cas d'utilisation : Récupération de la liste des logements

5. **Conclusion.**
   5.1 Résumé du projet
   5.2 Évolutions futures

1 Introduction du projet

1.1 Objectif du document :

Ce document a été rédigé dans le but de fournir une vision claire et complète de l'architecture et du design de l'application HBnB.
Il servira de guide tout au long de la réalisation du projet et, en cas de difficultés, il suffira de s'y référer, car il constitue une véritable feuille de route pour l'implémentation et la maintenance.

1.2 Présentation des différentes couches :

Ce document couvre l'ensemble des aspects essentiels de l'architecture et du design de l'application HBnB. Il présente d'abord l'architecture à trois couches, en détaillant la manière dont les différentes couches interagissent entre elles grâce au pattern Facade.

Il inclut ensuite les diagrammes de classes de la logique métier, illustrant les principales entités du système, leurs attributs, méthodes et relations.
Enfin, il présente les diagrammes de séquence pour les appels API principaux, permettant de visualiser le flux d'informations entre la couche de présentation, la logique métier et la persistance des données.

L'objectif est de fournir une vision complète et compréhensible qui servira de référence tout au long du projet.

1.3 Hbnb, c'est quoi ?:

Notre site web **Hbnb** s'inspire de la plateforme **Airbnb**. Nous souhaitons offrir aux utilisateurs la possibilité de louer ou de mettre en location leurs biens immobiliers. La plateforme intégrera également un système d'avis laissés par les utilisateurs, ainsi que des fonctionnalités permettant d'effectuer des recherches précises sur différentes destinations, ou équipement.

2 Architecture Globale:

2.1 Description de la structure :

La structure Hbnb à été pensé pour reposer sur une architecture en trois couches ou niveaux

1. **La couche de Présentation :**C'est la couche la plus haute de la plateforme, elle interagit directement avec l'utilisateur. On y retrouve les services exposer à l'utilisateur comme la réservation, la gestion de compte

2. **La couche de Logique Métier :** C'est la couche qui définit les modèles ainsi que les règles dans BusinessRules qui indique ce qui est possible ou non.
   La méthode HbnbFacade permet de faire la liaison avec l'API, en effet Facade contient les méthodes simples ce qui permet de les modifier sans toucher à l' API.

3. **La couche de Persistance :**C'est la couche qui gère l'accès aux données , comment elles sont stockées, lues, modifiées ou supprimées.
   On y retrouve à l'intérieur les classes spéciales comme User, ainsi que les dossiers importants du système (DAO)

2.2 Description et choix de façade :

Pour faciliter les échanges entre ces couches, nous utilisons le **pattern Facade** : il agit comme une interface unifiée qui centralise les interactions. Ainsi, la couche de présentation n'a pas besoin de connaître les détails internes de la logique métier ou de la persistance, elle passe uniquement par la façade.
Ce choix simplifie non seulement le développement, mais rend également l'application plus robuste et plus facile à faire évoluer.

2.3 Diagramme d'architecture :

Le diagramme met en évidence l'organisation en trois couches de l'application et la manière dont elles communiquent entre elles.

- Les flèches indiquer par "**uses**" montrent que chaque couche dépend uniquement de la couche immédiatement inférieure, ce qui garantit une structure claire et modulable aux besoins.

- La **couche de présentation** correspond à la partie visible de l'application pour l'utilisateur. Elle expose les **services et points d'entrée (API endpoints)** qui permettent aux clients d'interagir avec le système, *comme l'inscription d'un utilisateur, la création d'une annonce ou la recherche d'un logement.*
  Cette couche se limite à recevoir les requêtes, les transmettre à la façade de la logique métier et renvoyer la réponse à l'utilisateur, sans contenir de règles métier en elle-même

- La présence de **HbnbFacade** dans la couche logique métier joue un rôle central car elle représente une interface unique entre la couche supérieure et la logique interne. Elle simplifie les appels des API et accélère la vitesse de traitement de la demande de l'utilisateur.

  Les **BusinessRules** sont des règles qui définissent et contrôlent le fonctionnement de l'application. Elles garantissent que toutes les actions respectent les contraintes et la logique du système.

  Exemple : un utilisateur ne peut laisser un avis que s'il a déjà effectué une réservation sur le logement concerné.

- La couche de persistance, illustrée par les **DAO**, agit comme un intermédiaire technique spécialisé.
  Les DAO sont des classes regroupées dans un dossier dédié, chargées d'assurer l'accès aux données. Grâce à ce mécanisme, la logique métier n'interagit jamais directement avec la base de données.

3 Logique Métier (Business Logic Layer) :

La couche de logique métier est le cœur de l'application **HBnB**. Elle définit les différents modèles ainsi que leurs interactions, en se basant sur les besoins réels du système.Cette partie a été conçue en respectant les principes **SOLID**, ce qui permet d'obtenir une architecture plus claire, modulaire et facile à maintenir.

3.1 Description des classes principales :

Le diagramme de classes présenté ci-dessus illustre les principales entités de l'application : **User**, **Place**, **Review**, **Amenity, BaseModèle** et **PlaceAmenity**.
Chaque classe possède des attributs (*comme le nom, l'email, le prix ou la description*) et des méthodes (*comme créer, mettre à jour ou supprimer une offre*). Ces méthodes représentent les actions que l'utilisateur peut effectuer via la plateforme.

Pour rentrer plus dans le détails :

- La classe **BaseModèle** est parents des autres classes, elle leur transmet les attributs de dates de création et de mise à jour ainsi que les clés ID.
- La classe **User** permet de créer ou de mettre à jour un profil.
- La classe **Place** gère les offres de logement, en incluant la recherche par localisation ou la mise à jour des informations.
- La classe **Review** centralise les avis laissés par les utilisateurs et les notes associées.
- La classe **Amenity** représente les équipements disponibles (Wi-Fi, parking, etc.), avec la possibilité de les ajouter ou de les supprimer.
- Enfin, la classe **PlaceAmenity** fait le lien entre un logement et ses équipements.

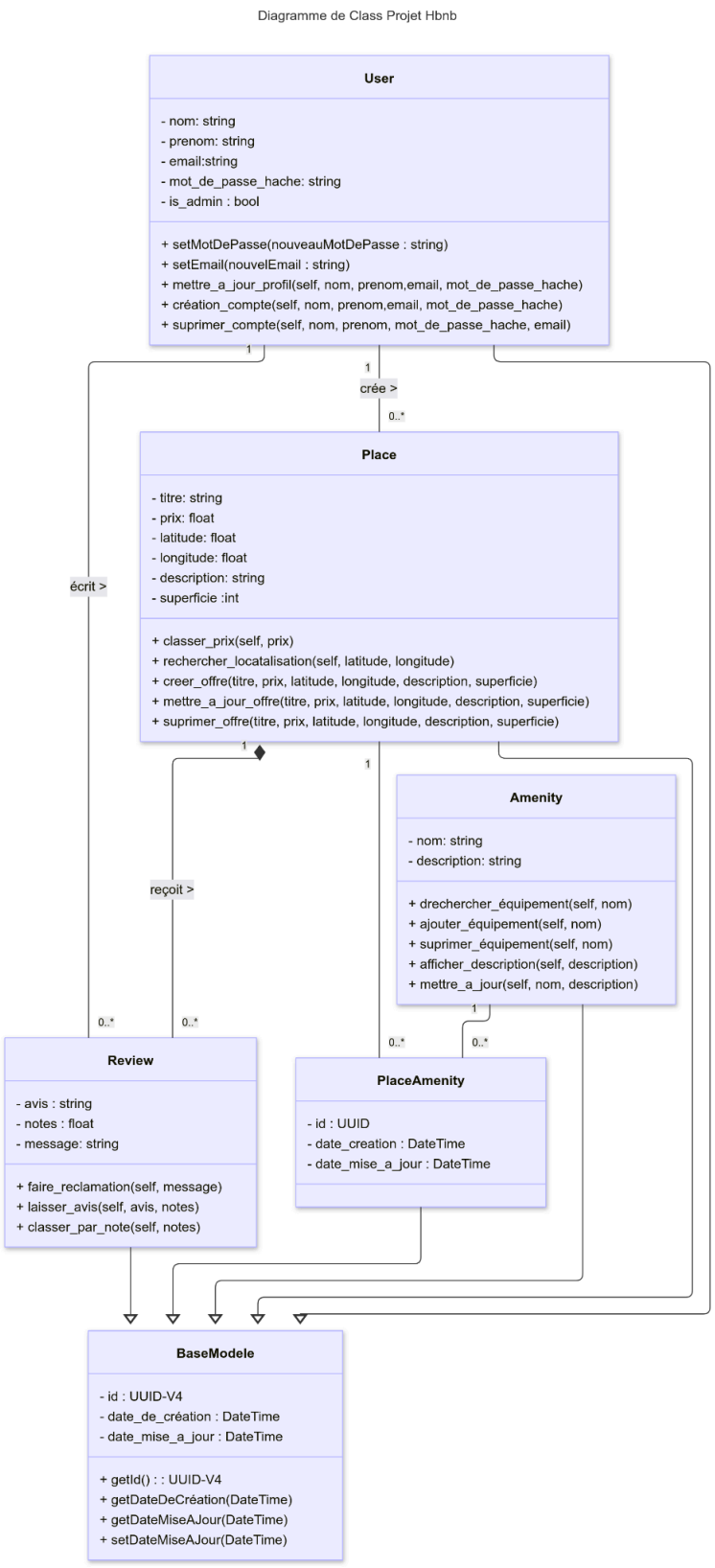
3.2 La cardinalité, les liens de relations :

Les **cardinalités** présentes dans le diagramme indiquent combien d'instances d'une classe peuvent être liées à une autre. Elles sont essentielles pour comprendre la structure des données.
Par exemple : *un utilisateur peut créer plusieurs logements, mais chaque logement appartient toujours à un seul utilisateur*.

De même, un logement **(Place)** peut recevoir plusieurs avis (**Review**), ce qui traduit le lien de dépendance entre ces deux classes : un avis ne peut exister que s'il est rattaché à un logement précis.

Une **relation de dérivation** apparaît lorsqu'il existe une association de type *n pour n* entre deux entités.Dans ce cas, on introduit une classe intermédiaire pour gérer cette relation.
C'est ce que montre la classe **PlaceAmenity**, qui fait le lien entre les logements et leurs équipements:
En effet un logement peut avoir plusieurs équipements, et un même équipement peut être associé à plusieurs logements.

3.3 Diagramme de classes UML :



3.4 Piste d'évolutions de notre diagramme :

Le diagramme de classes présenté constitue une première version fonctionnelle de l'architecture de l'application **HBnB**. Cependant, certaines améliorations peuvent être envisagées afin de rendre le système plus complet.
Une piste d'évolution importante serait l'ajout d'une classe **Réservation**, permettant de gérer le lien entre un utilisateur et un logement en intégrant des informations comme les dates de séjour et le prix associé.
De plus, l'introduction d'une classe **Message** offrirait la possibilité de traiter plus efficacement les réclamations et la communication entre les utilisateurs et les propriétaires, en établissant une liaison directe entre la classe **User** et la classe **Place**.

4 Appels des API (API Interaction Flow) :

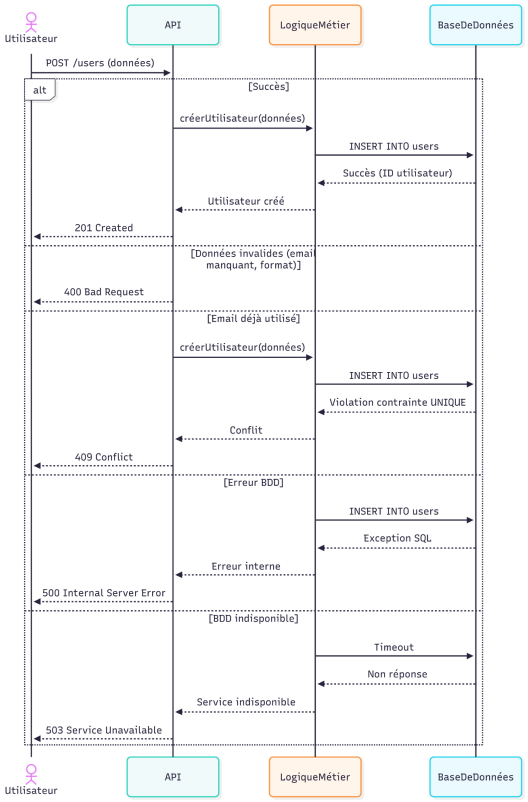
4.1 Méthodologie (diagramme de séquence) :

Les diagrammes de séquence illustrent le déroulement des interactions entre les différents éléments du système au cours d'un scénario précis.
Ils mettent en avant l'ordre chronologique des échanges, depuis l'action initiée par l'utilisateur jusqu'à la réponse générée par l'application. Grâce à ces représentations, on peut mieux comprendre le chemin parcouru par une requête, ainsi que le rôle joué par chaque couche de l'architecture.

4.2 Cas d'utilisation : Enregistrement utilisateur :

Le diagramme ci-dessus représente le cas où un utilisateur souhaite créer un compte en fournissant ses informations personnelles (nom, email, mot de passe).

- L'utilisateur envoie une requête POST /users/register avec ses informations (nom, prénom, email, mot de passe).
- L'API reçoit la requête et transmet les données à la logique métier.
- Si les règles sont respectées, la logique métier demande à la base de données d'enregistrer le nouvel utilisateur.
- L'API retourne une réponse de confirmation avec un 201 Created et les informations du compte.



Codes Erreurs :

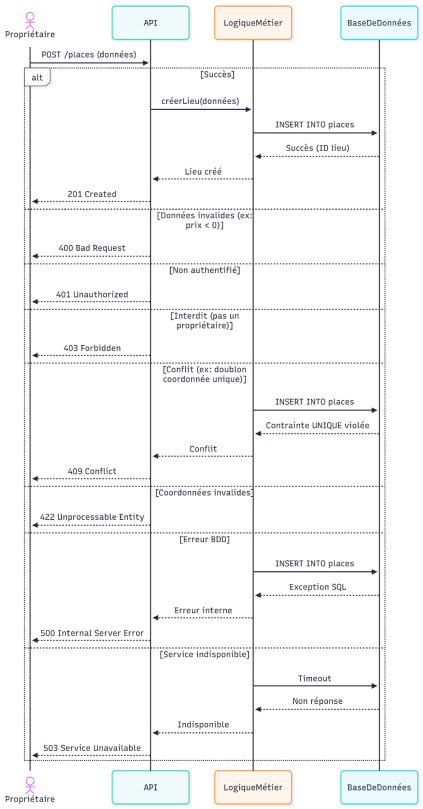
Le diagramme montre les validations côté logique métier (rectangles verticaux pour les traitements).
Ce scénario est ponctuel (pas de loop). Les codes possibles :

- **201** Créé avec succès.
- **400** Données invalides,
- **409** Conflit dans la demande,
- **422** Problème de connexion avec le serveur.
- **500** Site temporairement inaccessibles.
- **503** Serveur en maintenance.

4.3 Cas d'utilisation : Création d'une annonce :

Le diagramme ci-dessus représente le cas où un utilisateur authentifié souhaite publier une nouvelle annonce de logement avec ses détails (titre, prix, localisation, description).

- ➢ L'utilisateur envoie **POST /places** avec les informations du logement.
- ➢ L'API relaie la requête à la **logique métier**.
- ➢ La logique métier vérifie les **BusinessRules** (ex. : prix > 0, propriétaire valide).
- ➢ Si les contrôles passent, la logique métier enregistre la nouvelle **Place** via la **base de données**.
- ➢ L'API confirme la création avec **201 Created** et l'ID du logement.



Codes Erreurs :

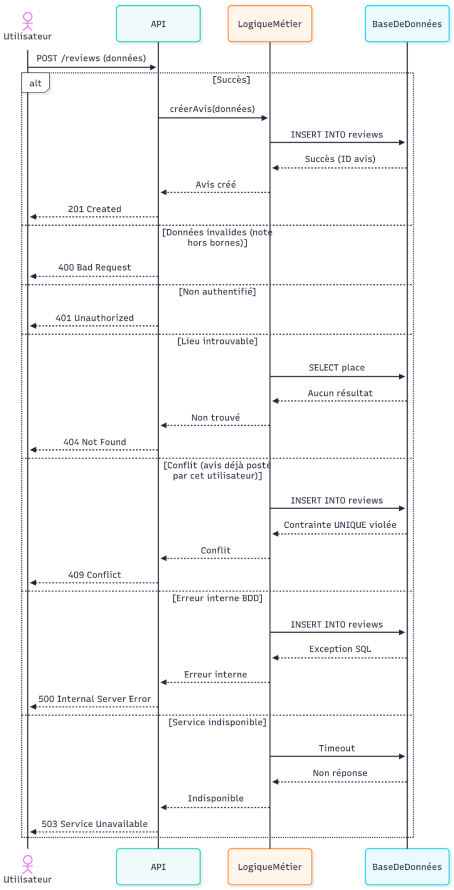
Ici le diagramme insiste sur la dépendance avec la classe *User* (seul un utilisateur connecté/propriétaire peut créer une annonce). Les rectangles montrent validation et persistance. Les codes possibles :

- **201** Créé avec succès.
- **400** Données invalides.
- **401** Accès à une page non autorisée.
- **403** Refusé au client par le serveur.
- **422** Problème de connexion avec le serveur.
- **500** Site temporairement inaccessibles.
- **503** Serveur en maintenance.

4.4 Cas d'utilisation : Soumission d'un avis :

Le diagramme ci-dessus représente le cas où l'utilisateur souhaite poster un avis et une note pour un logement qu'il a utilisé.

- ➢ L'utilisateur envoie **POST /reviews** (commentaire, note, id_ place, id_user).
- ➢ L'API transmet la requête à la **logique métier**.
- ➢ La logique métier applique les **BusinessRules** (ex. : vérifier que l'utilisateur a réservé le logement).
- ➢ Si la vérification est positive, la logique métier sauvegarde l'avis dans la **base de données**.
- ➢ L'API renvoie **201 Created** avec l'avis créé.



Codes Erreurs :

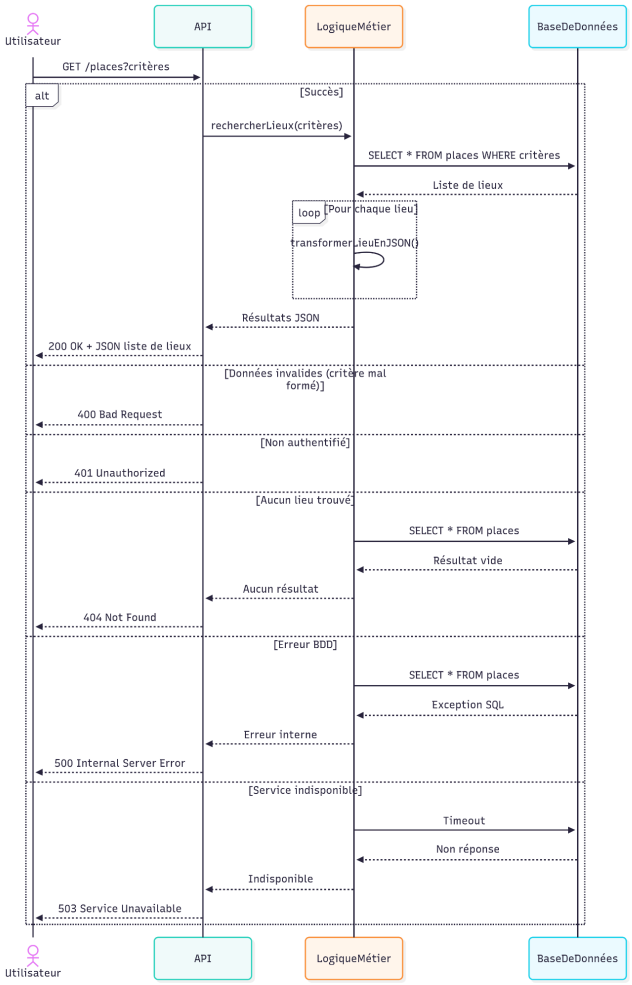
Ce scénario met l'accent sur une vérification préalable (rectangles pour validation puis sauvegarde). Contrairement aux autres, il peut renvoyer :

- **201** Créé avec succès.
- **403** Refusé au client par le serveur.
- **400** Données invalides.
- **401** Accès à une page non autorisée.
- 409 Conflit dans la demande.
- **500** Site temporairement inaccessible

4.5 Cas d'utilisation : Récupération de la liste des logements :

Le diagramme ci-dessus représente le cas où l'utilisateur l'utilisateur souhaite consulter les logements disponibles correspondant à ses critères (ou simplement voir tous les logements).

- ➢ L'utilisateur envoie une requête **GET /places** à l'API.
- ➢ L'API transmet la demande à la **couche de logique métier**.
- ➢ La logique métier interroge la **base de données** pour obtenir les logements.
- ➢ Les résultats sont transformés en objets **Place**.
- ➢ L'API renvoie la liste complète avec une réponse **200 OK**.



Codes Erreurs :

Le flux d'interaction se lit de gauche à droite via les flèches. Les **rectangles verticaux** montrent la durée d'une opération par une couche, et la **loop** indique la répétition (construction d'un objet *Place* par logement retourné).
Le diagramme mentionne aussi les codes HTTP :

- **200** Créé avec succès.
- **404** Erreur le serveur n'a pas trouvé la ressource demandée.

5. Conclusion :

5.1 Résumé du projet :

Le projet **HBnB** reprend les fondements d'une architecture logicielle moderne basée sur un modèle en trois couches : **présentation**, **logique métier** et **persistance des données**.
Grâce à l'utilisation du **pattern Facade**, les interactions entre ces couches sont simplifiées et clairement séparées, ce qui garantit une meilleure lisibilité du code, une maintenabilité accrue et une modularité permettant de faire évoluer la plateforme sans refonte complète.

La documentation présentée a permis de :

- Décrire l'architecture et le rôle de chaque couche.
- Définir les principales entités du domaine et leurs relations.
- Illustrer le fonctionnement via des **diagrammes UML** (classes et séquences).
- Détailler le cycle de vie des appels API en listant les scénarios d'utilisation et leurs codes d'erreurs associés.

Cette base constitue une **feuille de route technique** solide pour l'implémentation et la maintenance du projet HBnB.

5.2 Évolutions futures :

Afin d'enrichir l'application et de la rapprocher encore davantage des standards du marché, plusieurs améliorations peuvent être envisagées :

1. **Ajout de la gestion des réservations**
   - Introduction d'une classe *Réservation* liant un utilisateur et un logement, incluant dates, prix et statut.
2. **Messagerie interne**
   - Mise en place d'une classe *Message* permettant la communication entre hôtes et voyageurs (gestion des réclamations et demandes spéciales).
3. **Système de paiement sécurisé**
   - Intégration d'une API tierce (Stripe, PayPal…) pour gérer les transactions.
4. **Authentification avancée**
   - Mise en place d'un système OAuth2 (connexion via Google, Facebook, GitHub, etc.).
5. **Recherche et filtrage avancés**
   - Ajout de critères dynamiques (prix, localisation, équipements, disponibilités).